

Übungsblatt 5

Ü 5.1 **argc und argv**

Mithilfe der Parameter `argc` und `argv` in `int main(int argc, char *argv[])`, kann auf Werte die beim Aufruf des Programmes übergeben werden, beispielsweise über die Kommandozeile, zugegriffen werden.

Schreiben Sie ein Programm `argreader.c` welches eine beliebige Anzahl an Parametern akzeptiert. Zuerst soll es die Anzahl der erhaltenen Parameter ausgeben, dann sollen die erhaltenen Werte, jeweils in einer eigenen Zeile, ausgegeben werden.

Erweitern Sie das Programm schließlich so, dass es erkennt, wenn es sich bei einem Parameter um eine Zahl handelt, und falls dies so ist, noch zusätzlich die Summe aller bisher erhaltenen Zahlen daneben ausgibt. Testen Sie Ihr Programm mit unterschiedlichen Parametern!

Beispiel: Aufruf via `./argreader 1 2 nan 4`

Ausgabe: Anzahl der Parameter: 4
 1 Summe: 1
 2 Summe: 3
 nan
 4 Summe: 7

Ü 5.2 **exec**

Machen Sie sich mit folgenden Funktionen vertraut: `exec`-Familie

Schreiben Sie ein C-Programm `myexec` (siehe `myexec_stub.c`), das einen minimalistischen Kommando-Interpreter implementiert. Das Programm nimmt als Inputparameter einen Kommandonamen und dessen Inputparameter und führt das Kommando mit Hilfe einer Funktion der `exec`-Programmfamilie aus.

Beispielsweise sollte der Aufruf `./myexec ls -al` den Befehl `ls -al` innerhalb von `myexec` ausführen. Welcher Befehl der `exec`-Familie erscheint Ihnen am geeignetsten und warum?

Ü 5.3 Interprozesskommunikation: Pipes

Machen Sie sich mit folgender Funktion vertraut: `pipe`

Erweitern Sie den zur Verfügung gestellten Quellcode (`mypipe_stub.c`) wie folgt:

Schreiben Sie ein C-Programm, das aus einem Eltern- und einem Kind-Prozess besteht, die über Pipes kommunizieren. Der Kind-Prozess schickt dem Eltern-Prozess zuerst eine *variable* Anzahl von einzelnen `int`-Werten (erzeugt durch `rand()`), die der Eltern-Prozess bearbeitet und das Ergebnis am Bildschirm ausgibt. Die Bearbeitung besteht darin, den Durchschnitt der einzelnen Werte zu berechnen. Überlegen Sie sich, auf welche Weise man dem Parent die Anzahl der Werte kommunizieren könnte.

Ü 5.4 Interprozesskommunikation: Daemon

Erklären Sie, was man im Unix-Kontext unter einem „Daemon“ versteht, und welche Eigenschaften ihn auszeichnen.

Schreiben Sie ein C-Programm `daemon.c`, welches nach dem Start als Daemon läuft. Verwenden Sie `syslog()` um auszugeben und zu bestätigen, dass der Daemon läuft, und finden Sie somit auch die Prozess ID des Daemons heraus.

Wie können Sie den Daemon Prozess beenden?

Ü 5.5 Interprozesskommunikation: Signale

Erklären Sie, was ein Signal im Unix-Kontext ist und machen Sie sich mit folgender Funktion vertraut: `signal`.

Erweitern Sie `daemon.c`, welches nach dem Start des Daemons, Signale an den Daemon sendet. Erhält der vorhin gestartete Daemon das Signal "SIGUSR1", soll „SIGUSR1 received“ per `syslog()` ausgegeben werden. Wird das Signal erneut erhalten, soll der Daemon-Prozess beendet werden. Versehen Sie Ihr C-Programm dazu mit einem Signalhandler.